

Preguntas y Respuestas Frecuentes sobre Docker y Entornos de Desarrollo

1. ¿`docker compose up` reemplaza a `docker build`?

No exactamente. `docker compose up` puede incluir el proceso de `build` si el servicio está configurado con `build:` en el `docker-compose.yml`. Es decir, `docker compose up` construye y levanta los contenedores en una sola instrucción.

2. ¿La versión 3.7 del `docker-compose` es la última?

No. La versión 3.8 es más nueva que la 3.7. Actualmente, se recomienda usar `3.8` o la versión más reciente soportada por tu instalación de Docker.

3. ¿Para qué sirven los volúmenes (`volumes`) en Docker?

Los volúmenes sirven para persistir datos fuera del contenedor. Así, aunque elimines o reinicies el contenedor, los datos (como los de MongoDB) se mantienen.

4. ¿Docker usa las variables `.env` de mi app o solo las suyas?

Por defecto, Docker usa variables definidas en su propio contexto (`docker-compose.yml` o `env\_file`). Tu app puede seguir usando su propio `.env`, pero si el contenedor no lo copia o no lo accede, no lo verá.

5. ¿Es necesario usar la variable `MONGO\_URI` si ya tengo `mongo` definido en Docker Compose?

Depende. Si tu app se conecta a MongoDB, necesitas pasarle una `MONGO\_URI`. Puedes usar algo como: `mongodb://usuario:clave@mongo:27017`. Aquí "mongo" es el nombre del servicio, y Docker lo resuelve automáticamente.

6. ¿Dónde defino las variables de entorno (`env`)?

Puedes definir las en: el archivo `.env`, directamente en `docker-compose.yml` bajo `environment`, o en el `Dockerfile` (aunque esto es menos común para datos sensibles).

7. ¿Es obligatorio usar `env\_file` si me funciona sin él?

No es obligatorio, pero es más ordenado y reutilizable. ``env_file`` te permite mantener las variables en un lugar separado y limpio, y evitar repetirlas.

8. ¿``env_file`` guarda mis valores o solo los referencia?

Solo los referencia. Docker carga ese archivo al momento de levantar los servicios, pero no lo modifica ni guarda cambios ahí.

9. ¿Cuáles son las ventajas de poner variables en Docker Compose?

Separas configuración del código, facilitas el despliegue en diferentes entornos, y puedes mantener una sola fuente de verdad para las variables.

10. ¿Me evito usar ``dotenv`` si pongo las variables en Docker Compose?

Sí, si pasas todas las variables necesarias como ``environment`` o con ``env_file``, no necesitas ``dotenv`` en el código.

11. ¿Es más conveniente levantar el proyecto con Docker o desde terminal en desarrollo?

Para simplicidad rápida, terminal. Para coherencia y compatibilidad entre máquinas o equipos, Docker.

12. ¿Puedo usar Docker solo para asegurarme de tener las versiones correctas, y luego trabajar con terminal?

Sí. Puedes levantar los contenedores una vez para ver qué versiones se usan, instalarlas localmente, y luego trabajar sin Docker.

13. ¿El ``package.json`` no logra lo mismo?

No del todo. ``package.json`` gestiona solo dependencias de Node.js, no de herramientas externas como MongoDB o Redis. Docker define todo el entorno.

14. ¿Si clono una app que usa Docker, cómo la levanto y luego trabajo desde terminal?

Ejecuta ``docker compose up``, identifica las versiones usadas, instálalas en tu sistema si quieres, y luego trabaja desde tu terminal como normalmente.